

App Summary

Repo evidence summary for the current application surface

Naming note:

README says "Espace PMS"; frontend metadata/marketing present "CondoFlow".

WHAT IT IS

A multi-tenant SaaS for condominium and property operations. Repo evidence shows admin, resident, and superadmin workflows for billing, communication, maintenance, and reporting.

WHO IT'S FOR

Primary persona: condo association admins or property managers who need one system for buildings, residents, invoices, payments, issues, and resident communication.

WHAT IT DOES

- Manages buildings, apartments, residents, and team access.
- Handles invoices, payments, balances, subscriptions, and billing views.
- Supports announcements, documents, voting, and resident chat.
- Tracks issues, maintenance, suppliers, expenses, and reminders.
- Provides imports, reconciliation, reports, audit logs, and onboarding.
- Includes superadmin tools for organizations, leads, demos, jobs, and system status.

HOW IT WORKS

- Frontend: Next.js 14 (`frontend`) with `next-intl`, role-based routes, and a shared API client.
- Backend: NestJS (`backend`) with feature modules plus global JWT, RBAC, and organization-scope guards.
- Data layer: Prisma over PostgreSQL (`schema.prisma` + `docker-compose.yml`).
- Realtime/data flow: browser UI -> REST/WebSocket backend -> Prisma -> PostgreSQL; Socket.IO uses org-scoped rooms.

HOW TO RUN

- Install dependencies: `npm install`.
- Create env files: `cp backend/.env.example backend/.env` and `cp frontend/.env.example frontend/.env.local`.
- Start PostgreSQL: `docker compose up -d postgres`.
- Initialize DB: `cd backend && npx prisma migrate dev && npx prisma db seed`.
- Run both apps: `npm run dev`.
- Open `http://localhost:3001`; health: `http://localhost:4000/api/health`.

EVIDENCE GAPS

No major required summary item was missing. Branding is mixed across the repo.